hashlock.

Security Audit

Tapir Money (DeFi)

June 2026 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

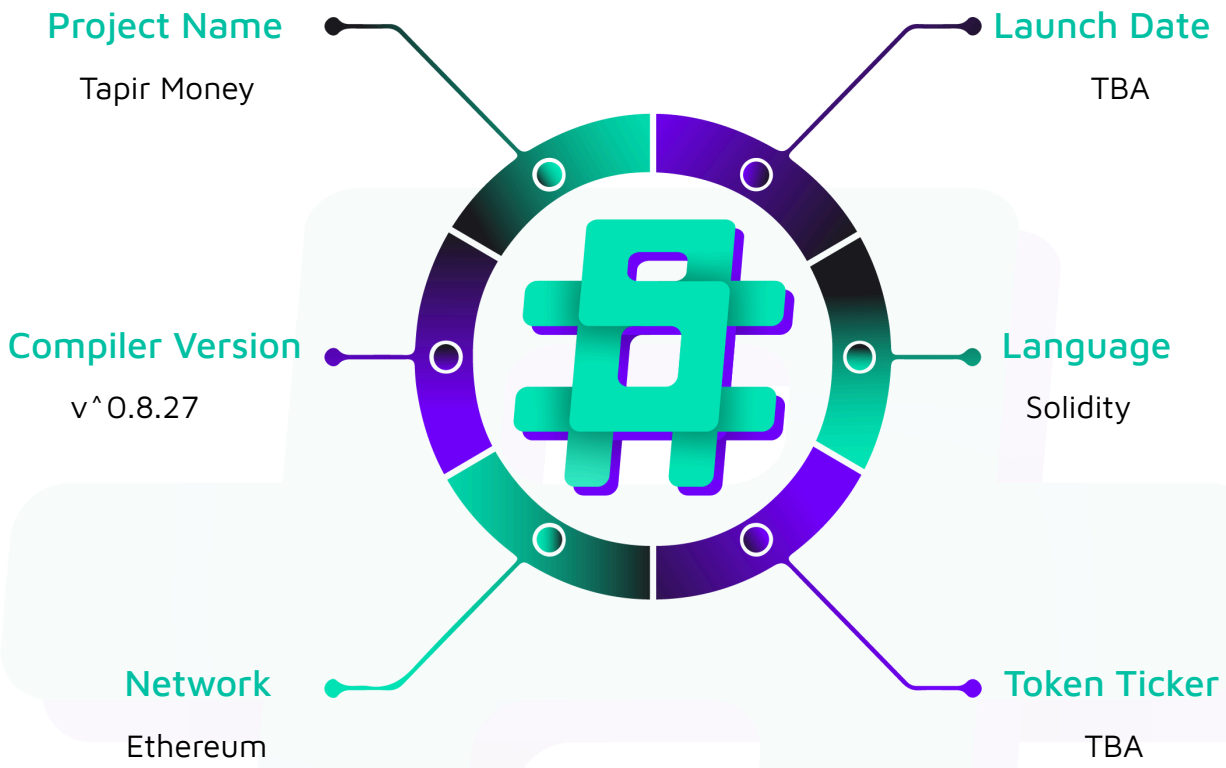
The Tapir Money team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

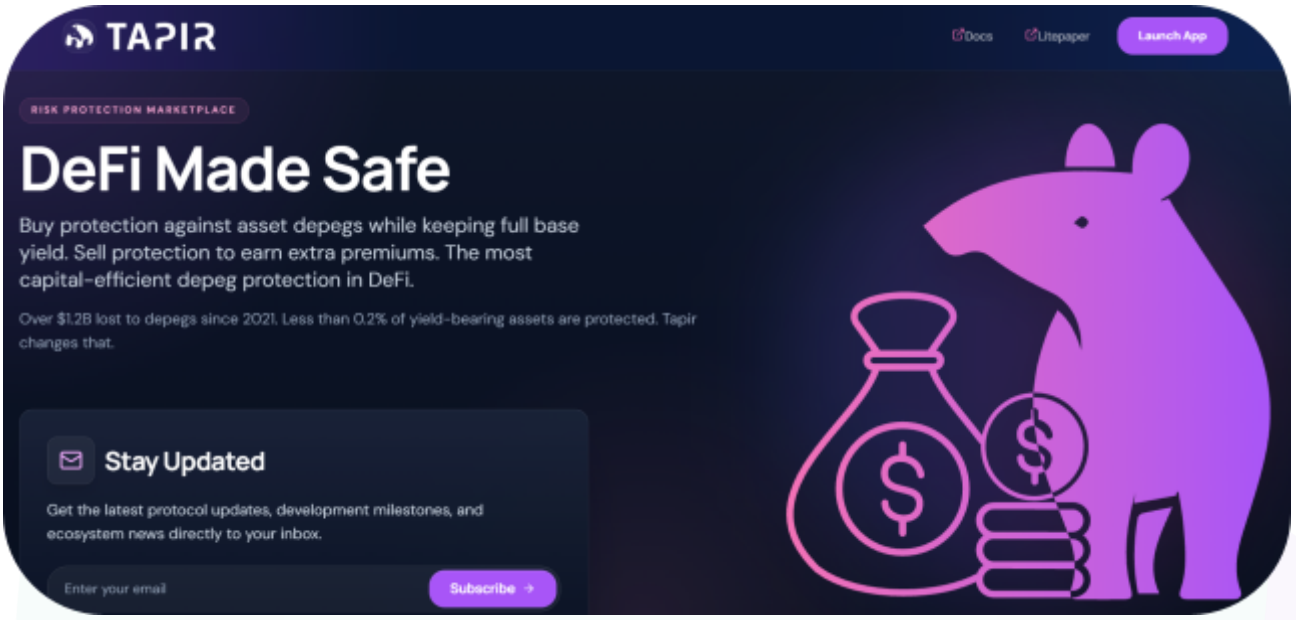
Tapir is a decentralized finance (DeFi) protocol focused on depeg risk management. It operates as a marketplace where users can either purchase protection against stablecoin or liquid staking token depegs or earn additional yield by selling that protection. Unlike traditional DeFi insurance models that require idle collateral, Tapir keeps capital productive by splitting yield-bearing assets into protection-focused and yield-boosted positions, allowing users to customize their risk exposure while continuing to earn underlying returns. The protocol aims to improve capital efficiency and make DeFi risk hedging more accessible for yield-seeking investors.

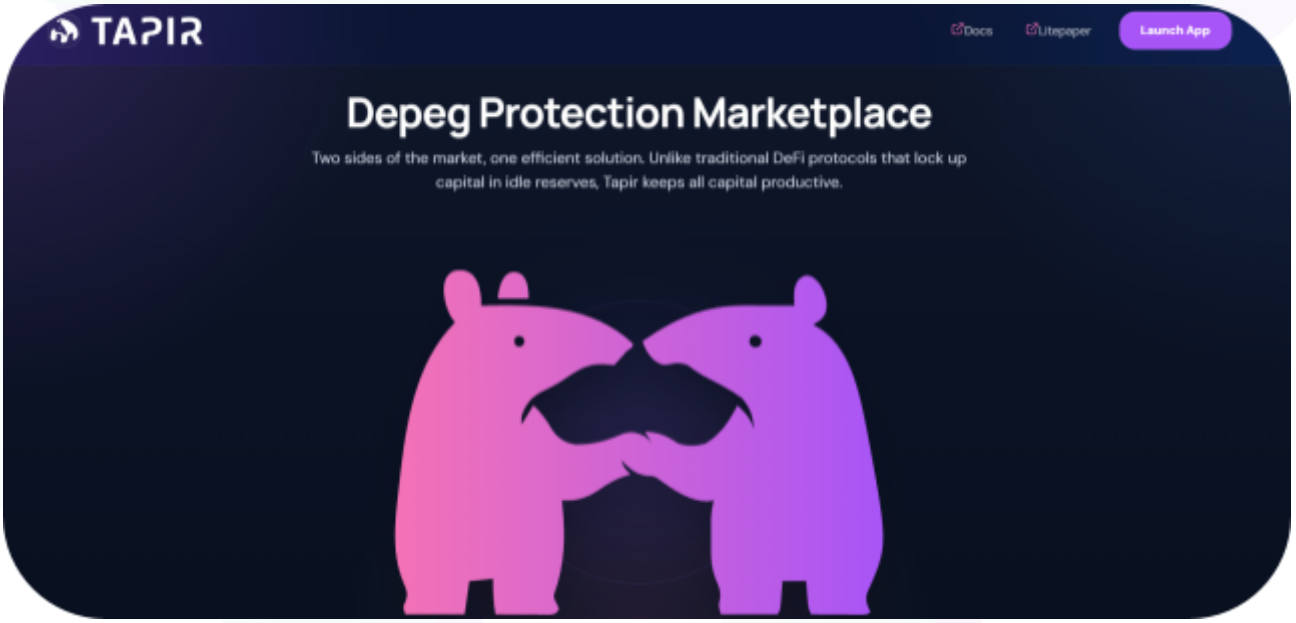
Project Name: Tapir Money
Project Type: Defi
Compiler Version: ^0.8.27
Website: https://tapir.money/
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the Tapir Money project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Tapir Money Smart Contracts**
Network	**Ethereum**
Language	**Solidity**
Audit Date	**June, 2026**
Contract 1	contracts/DepegFactory.sol
Contract 2	contracts/DepegPool.sol
Contract 3	contracts/TapirOracle.sol
Contract 4	contracts/TapirPtrwOracle.sol
Contract 5	contracts/TapirRouter.sol
Contract 6	contracts/TapirVrpOracle.sol
Contract 7	contracts/libraries/TimeDecay.sol
Audited GitHub Commit Hash	e951edc34939d8191d65c5ef29c2fc2e4ac5a462
Fix Review GitHub Commit Hash	9fc06de7ad6531771ab245e01828459d9b0b00a7

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerabilities

1 Medium severity vulnerability

3 Low severity vulnerabilities

2 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour

TapirOracle.sol
- Allows the admin to:
  - Configure price feed sources via setSources
  - Update oracle parameters (minValidSources, checkpointSpacing, maxStaleness) via setConfig
  - Pause and unpause checkpoint creation
- Allows the operator to:
  - Record fresh prices from each individual feed (Chainlink, API3, RedStone Classic, Tellor)
  - Commit a new price checkpoint aggregating valid, non-stale sources
  - Write the finalised HWM and closing price to the connected DepegPool
	Contract achieves this functionality.

TapirVrpOracle.sol
- Allows the operator to:
  - Record checkpoint alongside a VRP ring-buffer snapshot in a single call
  - Resolve VRP by sampling the vault's live previewRedeem at settlement time
  - Write VRP-adjusted HWM and closing price to the DepegPool
- Allows the admin to:
  - Manually supply a PRV value if the auto resolveVrp path returns zero
	Contract achieves this functionality.

(manualBackupResolveVrp)	

TapirPtrwOracle.sol
- Allows the operator to:
  - Resolve PTRW by redeeming PT tokens held in the oracle via the Pendle router, recording ERV and ARV
  - Write PTRW-adjusted HWM and closing price to the DepegPool
- Allows the admin to:
  - Manually supply ERV and ARV if the auto resolvePtrw path fails (manualBackupResolvePtrw)
	Contract achieves this functionality.

DepegPool.sol
- Allows anyone to:
  - Split base tokens into equal DP and YB tranches while the pool is ACTIVE
  - Unsplit DP and YB back into base tokens while the pool is ACTIVE
  - Trigger COOLDOWN state if the oracle price breaches the depeg threshold
  - Redeem DP tokens for their pro-rata payout once the pool reaches RESOLUTION
  - Redeem YB tokens for their pro-rata payout once the pool reaches RESOLUTION
- Allows the oracle to:
  - Write finalised HWM and closing price data during COOLDOWN, advancing the pool toward RESOLUTION
	Contract achieves this functionality.

- Allows the admin to: - Sweep accumulated split fees to the treasury address	

DepegFactory.sol
- Allows the admin to:
  - Deploy and register new DepegPool instances with validated oracle, treasury, and configuration parameters
  - Support cross-chain deployment mode where the oracle address may be an EOA rather than a contract
	Contract achieves this functionality.

TapirRouter.sol
Allows anyone to:
  - Split base tokens and immediately purchase a specific tranche (DP or YB) in a single transaction via splitAndBuy
  - Sell a tranche back and unsplit to base in a single transaction via sellAndUnsplit
  - Perform a direct token swap between supported assets via swapExactIn
	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Tapir Money project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Tapir Money project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] TapirVrpOracle#writePriceData - Un-smoothed single sample and missing role gate on `writePriceData` allow manipulation of settlement price

Description

`TapirVrpOracle.writePriceData` is declared `public` with no access-control modifier, whereas the structurally identical function in the `TapirPtrwOracle` contract is restricted to `OPERATOR_ROLE`. Once `vrpResolved` is set to `true` by the operator, any account — including an unprivileged attacker — can finalize settlement by calling `writePriceData`. Compounding this, the closing-price scaling component (`vrpPrv`) is drawn from a single, live, un-smoothed `previewRedeem` sample at `resolveVrp` time, with no floor protection on downward movement. If the production vault's `previewRedeem` is balance/NAV-sensitive, an attacker can collapse the live preview immediately before operator resolution, manufacture a false depeg, and self-finalize through the public entry point.

Vulnerability Details

`writePriceData` carries no role restriction:

```
function writePriceData(uint32 _gaslimit) public payable override {
```

This value flows directly into the closing-price scaling with only an upward cap — no downward floor:

```
uint256 effectiveClosingVrp = vrpPrv + errorTolerance < vrpHwmValue
    ? vrpPrv + errorTolerance
    : vrpHwmValue;
uint256 adjustedClosingPrice = Math.mulDiv(closingPrice, effectiveClosingVrp, initialVrpPreview);
```

The `min(vrpPrv + errorTolerance, vrpHwmValue)` expression caps upward drift but imposes no floor on a collapsed `vrpPrv`. A vault whose `previewRedeem` responds to asset balance (e.g. OZ-style balance-based ERC-4626 where `previewRedeem(shares) = shares × (totalAssets + 1) / (totalSupply + 1)`) can have its live preview artificially collapsed by a non-proportional asset outflow — assets removed without burning corresponding shares — executed as a single transaction immediately before `resolveVrp` is called.

Impact

An unprivileged attacker holding no protocol role can fabricate a depeg at settlement, transferring value from YB holders to DP holders. The closing price can be driven to approximately 25% of its honest value in a single transaction.

Recommendation

The most direct remediation is to restrict `writePriceData` to `OPERATOR_ROLE`, making it consistent with the PTRW oracle and eliminating unprivileged finalization entirely.

Status

Resolved

Medium

[M-01] TapirOracle#_highWatermarkPrice - Bounded checkpoint ring can forget the protection-epoch high-water mark

Description

`TapirOracle` derives the high-water mark (HWM) as the maximum of the minimum price across every consecutive three-day window. However, the accepted HWM is not persisted. `_highWatermarkPrice()` recomputes it only from observations remaining in a 400-entry circular buffer.

After the 401st checkpoint, `_pushPoint()` overwrites the oldest observation. If three early high-price days established the HWM, evicting the first high day leaves only two consecutive high days. No retained three-day window is then entirely high, causing the recomputed HWM to fall to the later low-price regime. This violates the required monotonic invariant:

```
HWM(after a valid checkpoint) >= HWM(before the checkpoint)
```

The issue is reachable through honest operation. The tested one-year schedule records three early high-price days, 360 daily low-price checkpoints, then 38 half-hour low-price checkpoints after the administrator enables the specification-supported near-maturity cadence. This produces 401 observations before resolution without malicious checkpoint spam or feed manipulation.

The same bounded-ring and recomputed-HWM architecture exists in `TapirVrpOracle`. The base-oracle path is proven end to end. The VRP path is a source-verified instance of the same root cause, but its complete scaling and settlement impact has not been independently reproduced.

Vulnerability Details

```
// contracts/TapirOracle.sol
uint256 public constant MAX_POINTS = 400;
```

```
PricePoint[MAX_POINTS] private _points;

function _highWatermarkPrice() internal view returns (uint256) {
    PricePoint[] memory points = _materialize();
    uint256[] memory dailyPrices = _aggregateToDailyPrices(points);

    if (dailyPrices.length < 3) revert InsufficientCheckpointsForHWM();

    uint256 maxOfMins = 0;
    for (uint256 i = 0; i <= dailyPrices.length - 3; i++) {
        uint256 tripletMin = Math.min(
            dailyPrices[i],
            Math.min(dailyPrices[i + 1], dailyPrices[i + 2])
        );
        if (tripletMin > maxOfMins) {
            maxOfMins = tripletMin;
        }
    }
    return maxOfMins;
}

function _pushPoint(uint192 price, uint64 ts) internal {
    _points[_head] = PricePoint(price, ts);
    _head = uint16((_head + 1) % MAX_POINTS);
    if (_count < MAX_POINTS) {
        _count++;
    }
}

// contracts/TapirVrpOracle.sol
uint256 public constant MAX_VRP_POINTS = 400;
VrpPoint[MAX_VRP_POINTS] private _vrpPoints;

function _pushVrpPoint(uint192 preview, uint64 ts) internal {
    _vrpPoints[_vrpHead] = VrpPoint(preview, ts);
    _vrpHead = uint16((_vrpHead + 1) % MAX_VRP_POINTS);
    if (_vrpCount < MAX_VRP_POINTS) {
        _vrpCount++;
```

```
    }
}
```

Proof of Concept

The PoC uses three independently deployed active sources, `minValidSources = 3`, and the real API3, Chainlink, and RedStone recording functions. It executes the production three-source median, 401 checkpoints, `TapirOracle.writePriceData()`, and `DepegPool.resolvePriceDepeg()` against an actual one-year pool. It does not write ring storage directly.

Add the following source mocks at `contracts/mock/TapirOracleProductionPathMocks.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.27;

interface ITapirCheckpointOracle {
    function recordApi3Price() external;

    function recordChainlinkPrice() external;

    function recordRedStoneClassicPrice() external;

    function checkpoint() external;
}

contract CurrentTimestampApi3Mock {
    int224 private _value;

    constructor(int224 initialValue) {
        _value = initialValue;
    }

    function setValue(int224 newValue) external {
        _value = newValue;
    }
```

```
    function read() external view returns (int224 value, uint32 timestamp) {
        return (_value, uint32(block.timestamp));
    }

    function latestRoundData()
        external
        view
        returns (uint80 roundId, int256 answer, uint256 startedAt, uint256 updatedAt,
uint80 answeredInRound)
    {
        return (1, int256(_value), block.timestamp, block.timestamp, 1);
    }

    function decimals() external pure returns (uint8) {
        return 18;
    }
}

contract TapirOracleCheckpointOperator {
    function recordAndCheckpoint(address oracle) external {
        ITapirCheckpointOracle(oracle).recordApi3Price();
        ITapirCheckpointOracle(oracle).recordChainlinkPrice();
        ITapirCheckpointOracle(oracle).recordRedStoneClassicPrice();
        ITapirCheckpointOracle(oracle).checkpoint();
    }
}
```

Add the following test at `test/TapirOracleProductionPath.test.ts`:

```
import {expect} from "chai";
import {ethers} from "hardhat";
import {time} from "@nomicfoundation/hardhat-network-helpers";

const DAY = 24 * 60 * 60;
const HALF_HOUR = 30 * 60;
const YEAR = 365 * DAY;
const FOUR_HOURS = 4 * 60 * 60;

function poolParams(asset: string, oracle: string, owner: string, treasury: string) {
```

```
    return {
        assetAddress: asset,
        dpMetadata: {name: "Production Path DP", symbol: "PP-DP"},
        ybMetadata: {name: "Production Path YB", symbol: "PP-YB"},
        oracle,
        poolActiveDuration: YEAR,
        name: "Production Path Pool",
        flag: "PP",
        redemptionFeeBp: 0,
        cooldownDuration: FOUR_HOURS,
        poolOwner: owner,
        minPrice: ethers.parseEther("0.5"),
        maxPrice: ethers.parseEther("2"),
        treasury,
        authorisedRouter: ethers.ZeroAddress,
        minPriceAge: FOUR_HOURS,
        xDomainMessengerL2: ethers.ZeroAddress,
    };
}

describe("TapirOracle production-path ring-wrap invariant", function () {
    this.timeout(600_000);

    it("suppresses an early sustained depeg after spec-compliant elevated sampling
wraps the ring", async function () {
        const [deployer, treasury] = await ethers.getSigners();
        const high = ethers.parseEther("2");
        const low = ethers.parseEther("1");

        const Asset = await ethers.getContractFactory("WtETHMock");
        const asset = await Asset.deploy();

        const Source = await ethers.getContractFactory("CurrentTimestampApi3Mock");
        const api3Source = await Source.deploy(high);
        const chainlinkSource = await Source.deploy(high);
        const redStoneSource = await Source.deploy(high);

        const Operator = await
```

```
ethers.getContractFactory("TapirOracleCheckpointOperator");
        const checkpointOperator = await Operator.deploy();

        // TapirOracle requires the pool address before DepegPool is deployed.
        const nextNonce = await
ethers.provider.getTransactionCount(deployer.address);
        const predictedPool = ethers.getCreateAddress({
            from: deployer.address,
            nonce: nextNonce + 1,
        });

        const sources = {
            api3ReaderProxyV1IsActive: true,
            api3ReaderProxyV1: await api3Source.getAddress(),
            api3ReaderProxyV1Decimals: 18,
            api3ReaderProxyV1MaxStaleness: DAY,
            chainlinkAggregatorV3IsActive: true,
            chainlinkAggregatorV3: await chainlinkSource.getAddress(),
            chainlinkAggregatorV3Decimals: 18,
            chainlinkAggregatorV3MaxStaleness: DAY,
            redStoneClassicIsActive: true,
            redStoneClassicAggregator: await redStoneSource.getAddress(),
            redStoneClassicDecimals: 18,
            redStoneClassicMaxStaleness: DAY,
            tellorIsActive: false,
            tellorAdapter: ethers.ZeroAddress,
            tellorDecimals: 18,
            tellorMaxStaleness: DAY,
        };

        const initialConfig = {
            minCheckpointSpacing: DAY,
            minValidSources: 3,
            closingPriceLookbackPeriod: DAY,
            depegPool: predictedPool,
            xChainMode: false,
            xDomainMessengerL1: ethers.ZeroAddress,
        };
```

```
const Oracle = await ethers.getContractFactory("TapirOracle");
const oracle = await Oracle.deploy(
    "PP-ASSET",
    await asset.getAddress(),
    18,
    sources,
    initialConfig,
    deployer.address,
);

const Pool = await ethers.getContractFactory("DepegPool");
const pool = await Pool.deploy(
    poolParams(
        await asset.getAddress(),
        await oracle.getAddress(),
        deployer.address,
        treasury.address,
    ),
);
expect(await pool.getAddress()).to.equal(predictedPool);

const operatorRole = await oracle.OPERATOR_ROLE();
await oracle.grantRole(operatorRole, await checkpointOperator.getAddress());

async function checkpointAfter(spacing: number) {
    const last = Number(await oracle.lastCheckpointTs());
    const latest = await time.latest();
    const next = last === 0 ? latest + 1 : last + spacing;
    await time.setNextBlockTimestamp(next);
    await checkpointOperator.recordAndCheckpoint(await oracle.getAddress());
}

// Three consecutive high-price days establish the robust 2.0 HWM.
await checkpointAfter(1);
await checkpointAfter(DAY);
await checkpointAfter(DAY);
```

```
        // Price honestly falls to 1.0 for 360 ordinary daily observations.
        await api3Source.setValue(low);
        await chainlinkSource.setValue(low);
        await redStoneSource.setValue(low);
        for (let i = 0; i < 360; i++) {
            await checkpointAfter(DAY);
        }

        // Honest admin enables the specification-supported 30-minute cadence.
        await oracle.setConfig({...initialConfig, minCheckpointSpacing: HALF_HOUR});
        await checkpointAfter(DAY);
        for (let i = 1; i < 38; i++) {
            await checkpointAfter(HALF_HOUR);
        }

        // 401 writes occurred, but the circular buffer retains only 400 points.
        expect(await oracle.checkpointsCount()).to.equal(400);

        // Enter COOLDOWN and write through the real oracle-to-pool path.
        const activeEnd = Number(await pool.startTime()) + YEAR;
        await time.setNextBlockTimestamp(activeEnd + 1);
        await oracle.grantRole(operatorRole, deployer.address);
        await oracle.writePriceData(0);

        const finalPrice = await pool.finalPriceData();
        expect(finalPrice.hwmPrice).to.equal(low);
        expect(finalPrice.resolutionPrice).to.equal(low);

        // Correct retained HWM=2.0 against closing=1.0 gives DP/YB=20000/0.
        // Evicted HWM=1.0 instead reports no depeg and leaves both at par.
        const resolutionStart = activeEnd + FOUR_HOURS;
        await time.setNextBlockTimestamp(resolutionStart + 2);
        await pool.resolvePriceDepeg();

        expect(await pool.poolHasDepegged()).to.equal(false);
        expect(await pool.depegSize()).to.equal(0);
        expect(await pool.dpValue()).to.equal(10_000);
        expect(await pool.ybValue()).to.equal(10_000);
```

```
    });
});
```

Run:

```
npx hardhat test --config hardhat.audit.config.ts
test/TapirOracleProductionPath.test.ts
```

Observed result:

```
1 passing
401 recordAndCheckpoint calls
poolHasDepegged = false
DP/YB = 10,000/10,000
```

The exact boundary and unchanged-configuration negative control are covered by:

```
npx hardhat test --config hardhat.audit.config.ts test/TapirOracleFreshReview.test.ts
\
  --grep "retains the high|does not wrap"
```

The tests confirm:

```
390 observations: HWM remains 2.0
400 observations: HWM remains 2.0
401st observation: HWM falls to 1.0
```

Relevant artifacts:

- `contracts/mock/TapirOracleProductionPathMocks.sol`
- `contracts/mock/TapirOracleRingBufferHarness.sol`
- `test/TapirOracleProductionPath.test.ts`
- `test/TapirOracleFreshReview.test.ts`
- `hardhat.audit.config.ts`

Impact

An established depeg can be hidden after the ring wraps, causing pool-wide tranche misallocation. The production-path PoC produces HWM and closing prices of `1.0/1.0`, so the pool records no depeg and values DP/YB at `10,000/10,000`. Retaining the correct

`2.0` HWM against the `1.0` closing price instead produces a 50% depeg and DP/YB values of `20,000/0`. DP holders therefore lose the protection payout they purchased while YB holders retain value that should transfer to DP.

Recommendation

Persist the protection-epoch HWM separately from the bounded recent-history ring. Finalize one representative value per completed UTC day, retain enough finalized daily values to evaluate the current three-day candidate, and update the stored HWM monotonically:

```
candidate = min(day0, day1, day2)
persistedHwm = max(persistedHwm, candidate)
```

Use the raw checkpoint ring only for recent-history and closing-price calculations. Apply the same architecture to the VRP HWM and add invariants proving that neither HWM can decrease at checkpoint 401 or any later checkpoint. Increasing `MAX_POINTS` alone is insufficient because capacity would remain uncoupled from pool duration and checkpoint cadence.

Status

Resolved

Low

[L-01] TapirPtrwOracle#function - Missing One-Shot Guard Allows Finalized ERV/ARV to Be Overwritten

Description

The PTRW oracle has two resolution paths: `resolvePtrw` (automatic, operator-driven) and `manualBackupResolvePtrw` (admin fallback). The manual path explicitly guards against re-resolution with `if (ptrwResolved) revert PtrwAlreadyResolved()`. The automatic path does not. Once a valid first resolution has set `ptrwErv`, `ptrwArv`, and `ptrwResolved = true`, a subsequent `resolvePtrw` call will succeed and overwrite those values — providd PT tokens are present in the oracle at the time of the second call. Because PT transfers are permissionless, anyone can re-seed the oracle's balance, and the operator may then — intentionally or inadvertently — trigger a second resolution against that new parcel.

Vulnerability Details

The asymmetry between the two paths is clear at the source level:

```
// contracts/TapirPtrwOracle.sol:77-82 — auto path: no ptrwResolved guard

function resolvePtrw() external onlyRole(OPERATOR_ROLE) {

    if (forceManualResolve) revert ManualResolveRequired();

    ptrwErv = IERC20(pendlePT).balanceOf(address(this));  // re-read on every call

    ...

}


// contracts/TapirPtrwOracle.sol:140 — manual path has the guard

if (ptrwResolved) revert PtrwAlreadyResolved();
```

When `resolvePtrw` runs a second time, it re-reads `ptrwErv` from the current PT balance (line 82) and overwrites `ptrwArv` (line 116) with the redemption yield of only

that second parcel. The original, protocol-representative measurement is silently discarded. At the extreme, a dust deposit where `errorTolerance >= newERV` erases any recorded haircut entirely, turning what was a genuine depeg result into a clean-redemption outcome.

Recommendation

Add the `ptrwResolved` check at the entry of `resolvePtrw`, reverting with the existing `PtrwAlreadyResolved` error.

Status

Acknowledged

[L-02] TapirOracle - Missing One-Shot Guard Allows Finalized ERV/ARV to Be Overwritten

Description

`checkpoint` admits any source whose cached price is non-zero and within the staleness window — it never checks `IsActive`. When an administrator disables a source via `setSources`, the `_sources` configuration is updated but the corresponding `latest*Price` slot is not cleared. The disabled feed continues contributing to quorum and price selection for up to `maxStaleness` seconds. The same issue fires on source swaps: the replaced feed's cached price persists alongside the new one.

Vulnerability Details

The checkpoint filter at `Line 280` checks only freshness and non-zero price, with no `IsActive` guard:

```
if (currentTs < priceSources[i].asOfTs + maxStaleness[i] && priceSources[i].price > 0) {
    validSources[validCount] = priceSources[i];
    validCount++;
}
```

An intentionally removed feed can independently satisfy quorum and influence HWM and closing price output for a full staleness window after the admin action.

Recommendation

The checkpoint filter should require `IsActive` for each source alongside the existing staleness check.

Status

Resolved

[L-03] TapirPtrwOracle - `resolvePtrw` Delegates Maturity Enforcement to the External Pendle Router

Description

`manualBackupResolvePtrw` guards against pre-maturity execution with a local `if (!IPendlePT(pendlePT).isExpired()) revert MarketNotExpired()` check. `resolvePtrw` does not — it proceeds directly from balance read to router call, relying on the external Pendle router to refuse the redemption if the PT has not yet expired. Today's Pendle router does enforce this, so the path is not currently reachable in practice. The issue is that the contract's own documented precondition is upheld by an external dependency rather than by the contract itself.

Vulnerability Details

The checkpoint filter at `Line 280` checks only freshness and non-zero price, with no `IsActive` guard:

```
// contracts/TapirPtrwOracle.sol:77-94 — no isExpired() check

function resolvePtrw() external onlyRole(OPERATOR_ROLE) {

    if (forceManualResolve) revert ManualResolveRequired();

    ptrwErv = IERC20(pendlePT).balanceOf(address(this));

    ...

     IERC20(pendlePT).approve(pendleRouter, ptrwErv);  // redeems with no local maturity
gate


// contracts/TapirPtrwOracle.sol:147 — manual path enforces it

if (!IPendlePT(pendlePT).isExpired()) revert MarketNotExpired();
```

Recommendation

Add the `isExpired()` check to `resolvePtrw`, bringing it in line with the manual path.

Status

Resolved

QA

[Q-01] TapirRouter - `sellAndUnsplit` distributes the router's full balance, making stranded tokens claimable by any caller

Description

`TapirRouter.sellAndUnsplit` computes its output amounts from `balanceOf(address(this))` rather than from the amounts produced by the current call. As a result, any tokens already stranded in the router — from a direct transfer, a residue from a failed interaction, or accidental deposit — are swept to `msg.sender` alongside the legitimate output of that call. Since any account can call `sellAndUnsplit`, an observer who notices a non-zero router balance can front-run to receive those stranded tokens.

Recommendation

The router's payout logic should be based on the difference between the balance before and after the current call's internal operations, rather than the absolute post-call balance.

Status

Acknowledged

[Q-02] TapirVrpOracle#writePriceData - VRP-scaled HWM exceeds `MAX_PRICE`, permanently locking the pool in COOLDOWN

Description

`TapirVrpOracle.writePriceData` scales the price high-watermark (HWM) by the live vault growth factor before forwarding it to `DepegPool.updatePriceData`. Because any yield-bearing ERC-4626 vault's `previewRedeem` grows monotonically over time, the scaling factor only increases. Once it has grown enough to push the adjusted HWM above `MAX_PRICE`, every call to `writePriceData` reverts inside the pool. Since `updatePriceData` is the only function that can advance the pool out of COOLDOWN, the pool becomes permanently stuck — all depositor funds are locked with no on-chain recovery path.

Vulnerability Details

`writePriceData` computes the adjusted HWM at `TapirVrpOracle.sol:138`:

```
uint256 vrpHwmValue = _vrpHighWatermarkPreview();

...

uint256 adjustedHwmPrice = Math.mulDiv(hwmPrice, vrpHwmValue, initialVrpPreview);
```

`initialVrpPreview` is the `previewRedeem(witnessShares)` snapshot taken once in the constructor (`TapirVrpOracle.sol:64`). For any yield-bearing vault `vrpHwmValue` monotonically increases relative to it, making `adjustedHwmPrice > hwmPrice` after any yield accrual. There is no upper clamp before the value is forwarded to the pool.

On the receiving side, `DepegPool.updatePriceData` is reachable only in COOLDOWN and enforces an immutable ceiling:

```
if (state != STATE_COOLDOWN) revert MustBeInCooldownState(state);

if (_hwmPrice < MIN_PRICE || _hwmPrice > MAX_PRICE) revert PriceOutOfBounds(_hwmPrice, MIN_PRICE, MAX_PRICE);
```

`MAX_PRICE` is set once at pool construction and cannot be changed. Once `adjustedHwmPrice > MAX_PRICE`, every `writePriceData` call reverts, `finalPriceData` is

never written, and `getState()` never advances past COOLDOWN. The revert is permanent because the only state-advancing function is permanently blocked.

`writePriceData` applies an unbounded growth multiplier to `hwmPrice` with no ceiling clamp, while the downstream pool enforces a hard, immutable `MAX_PRICE` ceiling. There is no on-chain coupling that forces `MAX_PRICE` to account for expected vault appreciation over the pool's lifetime.

Impact

All DP and YB holder funds in the affected pool are locked indefinitely. Splits are disabled in COOLDOWN state, `resolvePriceDepeg` cannot run until `finalPriceData` is written, and that write permanently reverts. The trigger is ordinary vault yield — no attacker and no admin error is required. Because `writePriceData` is a public function, any account can trigger the terminal revert once the growth threshold is crossed.

Recommendation

with the pool's accepted range. Concretely, the adjusted HWM should be bounded at the pool's `MAX_PRICE` rather than allowed to exceed it silently — the oracle should query the pool's immutable price bounds and clamp or handle the overflow before the cross-contract call. In parallel, deployment tooling or documentation should mandate that `MAX_PRICE` is provisioned with explicit headroom for the expected cumulative vault NAV growth over the full pool lifetime, with the current vault rate factored in at deploy time. Relaxing the HWM bound check specifically for VRP pools is also a viable design path, since the VRP-adjusted HWM is structurally unbounded by vault yield, whereas the original `MAX_PRICE` was designed to bound the unadjusted market price.

Note

This finding was raised at High severity in the original review of June 2026 and fixed in code during the fix review, with a clamp bounding adjustedHwmPrice to MAX_PRICE. The current classification reflects that merged fix together with the operating assumptions documented by the Tapir team, under which pools are initialised with sufficient MAX_PRICE headroom for non-malicious growth. Provisioning adequate headroom at pool initialisation remains an operational requirement.

Status

Acknowledged

Centralisation

The Tapir Money project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Tapir Money project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au